

FreeRTOS Core Concepts (Theory)

FreeRTOS Core Concepts (Theory)

- 1. What is RTOS
 - 1.1 Bare Metal vs RTOS
 - 1.2 Why Use RTOS
- 2. Task
 - 2.1 What is a Task
 - 2.2 Task State Machine
 - 2.3 Task Creation
 - 2.4 Task Lifecycle Control
 - 2.5 Task Diagnostics
- 3. Scheduler
 - 3.1 Scheduling Policy
 - 3.2 Priority
 - 3.3 Preemption Diagram
 - 3.4 Round-Robin Time Slice Diagram
 - 3.5 When Tasks Yield CPU
- 4. Inter-Task Communication
 - 4.1 Five Mechanisms Overview
 - 4.2 Queue — FIFO Data Pipeline
 - 4.3 Binary Semaphore — ISR → Task Notification
 - 4.4 Mutex — Mutual Exclusion Lock
 - 4.5 Event Group — 24-bit Bitmap
 - 4.6 How to Choose: A Diagram
- 5. Software Timer
- 6. ESP32-S3 FreeRTOS Features
- 7. Common Concurrency Problems
 - 7.1 Deadlock
 - 7.2 Priority Inversion
 - 7.3 Data Race
- 8. FreeRTOS Configuration in ESP-IDF
- 9. Learning Path Recommendations

1. What is RTOS

RTOS (Real-Time Operating System) is an operating system kernel designed specifically for embedded devices. Its core function is to **schedule multiple tasks according to priority and time constraints**, ensuring that critical tasks respond within specified time limits.

1.1 Bare Metal vs RTOS

Feature	Bare Metal (Super Loop)	RTOS
Program Structure	One big loop <code>while(1)</code> , executes sequentially	Multiple independent tasks, scheduler allocates CPU

Feature	Bare Metal (Super Loop)	RTOS
Response Latency	Depends on other code execution time in loop	Higher priority task preempts immediately when ready
CPU Utilization	One blocking (e.g., delay) → entire program stops	One task blocks → CPU automatically switches to other tasks
Complexity	Simple, suitable for single function	Requires understanding tasks, synchronization, priority
Code Maintainability	More features → "spaghetti code"	Each task is independent module, clear responsibilities

1.2 Why Use RTOS

Imagine a smartwatch — the following pseudocode comparison shows execution differences between bare metal and RTOS in the same scenario:

```
Bare Metal (Bare Metal) – all operations run sequentially in one loop:
while(1) {
    readHeartRateSensor();    // takes 50ms
    refreshScreen();          // takes 80ms
    wiFiUploadData();         // takes 200ms, blocks touch during upload
    detectTouchButton();      // takes 5ms
}

RTOS – each function is a separate task, scheduler assigns CPU by priority:
Task HeartRate (Prio 4):    read sensor every 100ms
Task Touch (Prio 3):        woken by touch, refreshes UI immediately
Task wiFi (Prio 2):         uploads data in idle time
Task Screen (Prio 1):       lowest priority, refreshes when CPU is free
→ touch always responsive, wiFi never blocks heart-rate reading
```

In bare metal programs, all code executes serially on a single timeline. RTOS splits the program into multiple independent tasks — **blocking operations no longer waste CPU, urgent events no longer wait in queue.**

2. Task

2.1 What is a Task

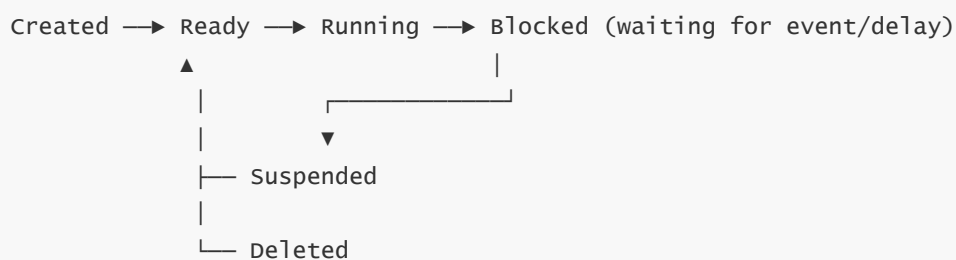
A task is the basic execution unit in FreeRTOS. Each task is a C function that never returns (infinite loop) with its own stack space.

Below is a minimal task skeleton: infinite loop + delay to yield CPU is the fixed pattern for FreeRTOS tasks.

```
// Task function, never returns
void vMyTask(void *pvParameters)
{
    // Every task must be an infinite loop
    while (1) {
        // do work
        // block for 1 second, yield CPU
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

2.2 Task State Machine

FreeRTOS tasks transition between five states. The following state diagram shows all possible transition paths:



State	Meaning	How to Enter
Ready	Ready, waiting for scheduler to allocate CPU	After task creation / waiting event arrived
Running	Currently executing	Only one per core at any time
Blocked	Blocked	<code>vTaskDelay</code> / waiting semaphore / waiting queue
Suspended	Suspended, not participating in scheduling	<code>vTaskSuspend()</code> call
Deleted	Deleted, memory reclaimed	<code>vTaskDelete()</code> call

2.3 Task Creation

`xTaskCreate` is the core API for creating tasks. Below is the complete function signature and parameter descriptions:

```
BaseType_t xTaskCreate(
    TaskFunction_t pvTaskCode,    // task function pointer
    // task name (for debugging, max 16 chars)
    const char * const pcName,
    // stack size in WORDS, not bytes
    uint32_t usStackDepth,
    // parameter passed to task
    void *pvParameters,
```

```

    // priority (higher = higher priority)
    UBaseType_t uxPriority,
    // task handle (pass NULL if not needed)
    TaskHandle_t *pxCreatedTask
);

// Pinned-to-core version
BaseType_t xTaskCreatePinnedToCore(..., BaseType_t xCoreID);
// Core 1 = APP_CPU (user application)

```

Stack size unit is WORDS, not bytes. On ESP32-S3, one word = 4 bytes. `usStackDepth = 2048` means $2048 \times 4 = 8192$ bytes = 8KB.

2.4 Task Lifecycle Control

API	Function	Notes
<code>vTaskSuspend(handle)</code>	Suspend	Task removed from Ready, no longer scheduled
<code>vTaskResume(handle)</code>	Resume	Must be called on suspended task
<code>vTaskDelete(handle)</code>	Delete	NULL = self-delete; cannot delete suspended tasks
<code>vTaskDelay(ticks)</code>	Relative delay	Calculated from call moment

2.5 Task Diagnostics

The following three diagnostic APIs are the most commonly used debugging tools during development — check priority, stack remaining, running core:

```

// Query current task priority
UBaseType_t prio = uxTaskPriorityGet(NULL);

// Query remaining stack space (Stack High Water Mark)
// Stack high water mark (words), smaller = riskier
UBaseType_t remaining = uxTaskGetStackHighWaterMark(NULL);

// Query which core the current task is running on
int core = xPortGetCoreID();

```

3. Scheduler

3.1 Scheduling Policy

FreeRTOS uses **fixed priority preemptive + same priority round-robin time slicing** scheduling. The following three rules summarize the core logic of the entire scheduling policy:

Rule 1 (Preemption):
Higher priority task ready → immediately preempts lower priority task

Rule 2 (Time Slice):
Same priority tasks ready → round-robin switch every tick

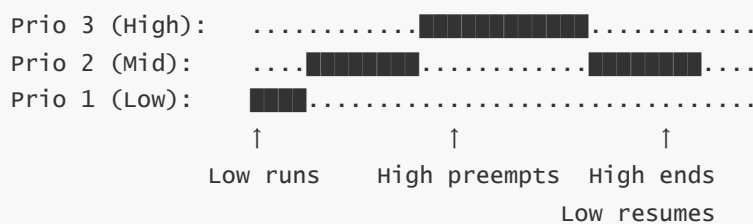
Rule 3 (Idle Fallback):
No task ready → IDLE task runs (priority 0)

3.2 Priority

- Value range: 0 ~ (configMAX_PRIORITIES - 1), default 0 ~ 24
- **Larger number = higher priority**
- IDLE task has fixed priority 0
- ESP-IDF main task (running `app_main`) has default priority 1

3.3 Preemption Diagram

The following timeline shows how high priority tasks immediately preempt low priority tasks:



3.4 Round-Robin Time Slice Diagram

Multiple tasks of the same priority alternate execution by time slice, switching every tick:



3.5 When Tasks Yield CPU

Operation	Description
<code>vTaskDelay()</code>	Active delay, task enters Blocked
<code>xSemaphoreTake()</code> / <code>xQueueReceive()</code>	Waiting for resource, task enters Blocked
Preempted by higher priority task	Passive yield, enters Ready
Same priority time slice expired	Passive yield, enters Ready

A high priority task that doesn't yield CPU will starve all lower priority tasks. High priority tasks must call `vTaskDelay` or blocking APIs at appropriate locations, otherwise the system essentially degrades to bare metal.

4. Inter-Task Communication

4.1 Five Mechanisms Overview

Mechanism	Purpose	Data Carried	Typical Scenario
Queue	Pass data between tasks	Any size data	Sensor data buffer, button events
Binary Semaphore	ISR → task event notification	None	GPIO interrupt notifies task to process
Mutex	Mutual exclusion for shared resources	None	Protect global variables, peripheral registers
Event Group	Multi-event synchronization	24-bit flags	Wait for multiple conditions simultaneously
Task Notification	Lightweight task wakeup	32-bit value	Single-to-single fast notification

4.2 Queue — FIFO Data Pipeline

FIFO data pipeline, standard way to pass data between tasks. The following code creates a queue with capacity 10 for unsigned 32-bit integers, and demonstrates basic send/receive usage:

```
// 10 slots, 4 bytes each
QueueHandle_t xQueue = xQueueCreate(10, sizeof(uint32_t));

// send, 100ms timeout
xQueueSend(xQueue, &data, pdMS_TO_TICKS(100));
// receive, block indefinitely
xQueueReceive(xQueue, &data, portMAX_DELAY);
```

Blocking vs Non-blocking:

<code>xTicksToWait</code>	Behavior when queue full/empty	Applicable Scenario
0	Return error immediately	Paths that cannot wait
<code>portMAX_DELAY</code>	Block forever until space/data available	Data must not be lost / event-driven receive

4.3 Binary Semaphore — ISR → Task Notification

Most commonly used for **ISR** → **task** notification. ISR is hardware interrupt service routine, cannot block, must complete quickly. Semaphore lets ISR only do "notification" and delegates actual processing to task.

```
// Initial value = 0
SemaphoreHandle_t xSem = xSemaphoreCreateBinary();
xSemaphoreGiveFromISR(xSem, &xHigherPriorityTaskWoken);

// inside task:
// block until ISR notification
xSemaphoreTake(xSem, portMAX_DELAY);
```

Must use `FromISR` version in ISR: `xSemaphoreGiveFromISR` / `xQueueSendFromISR` etc. Calling normal API in ISR will trigger assert.

4.4 Mutex — Mutual Exclusion Lock

Protect **shared resources**, ensuring only one task enters critical section at a time. Key differences from binary semaphore:

Feature	Mutex	Binary Semaphore
Purpose	Mutual exclusion — protect data	Synchronization — event notification
Priority Inheritance	✓	X
Who can give lock	Only the task that took it	Any task can give
Use in ISR	X	✓ (GiveFromISR)
Initial value	1 (available)	0 (blocking, waiting for event)

The following code uses mutex to protect increment operation of a global variable:

```
// Create mutex
SemaphoreHandle_t xMutex = xSemaphoreCreateMutex();

// Take lock - block if held
xSemaphoreTake(xMutex, portMAX_DELAY);
g_shared_var++; // ← critical section
xSemaphoreGive(xMutex); // Give lock back
```

4.5 Event Group — 24-bit Bitmap

24-bit bitmap, can wait for multiple events **all ready** or **any ready**. The following code creates an event group and waits for all event bits to be ready:

```
// Create event group
EventGroupHandle_t xGroup = xEventGroupCreate();
```

```
// Set event bits
// temperature data ready
xEventGroupSetBits(xGroup, BIT_TEMP_READY);
// pressure data ready
xEventGroupSetBits(xGroup, BIT_PRESS_READY);

// Wait for all events (AND)
xEventGroupWaitBits(xGroup,
    BIT_TEMP_READY | BIT_PRESS_READY, // which bits to wait for
    pdTRUE, // clear bits on exit
    pdTRUE, // pdTRUE=AND (all), pdFALSE=OR (any)
    portMAX_DELAY); // wait forever
```

4.6 How to Choose: A Diagram

This decision flowchart covers typical selection scenarios for five communication mechanisms, starting from "whether data needs to be passed":

```
Need to pass data?
├ Yes → Queue
└ No → Need to protect shared resource?
    ├── Yes → Mutex
    └ No → Need to wait for multiple events?
        ├── Yes → Event Group
        └ No → ISR notifying task?
            ├── Yes → Binary Semaphore
            └ No → Task Notification (faster)
```

5. Software Timer

Unlike hardware timers, FreeRTOS software timers are managed by the kernel's **Timer Service Task**. Callback functions execute in the timer service task context (not ISR context), so most FreeRTOS APIs can be called in callbacks.

```
// Create a software timer
TimerHandle_t timer = xTimerCreate(
    "MyTimer", // timer name
    pdMS_TO_TICKS(100), // 100ms period
    pdTRUE,
    0, // timer ID
    vTimerCallback // callback function
);

xTimerStart(timer, 0); // Start the timer
```

<code>uxAutoReload</code>	Behavior	Typical Scenario
<code>pdTRUE</code>	Periodic triggering, never stops	LED breathing, sensor timed polling

<code>uxAutoReload</code>	Behavior	Typical Scenario
<code>pdFALSE</code>	Stops automatically after one trigger	Timeout protection, delayed operation

6. ESP32-S3 FreeRTOS Features

Feature	Parameters
CPU Cores	2 (Core 0 = PRO_CPU, Core 1 = APP_CPU)
Tick Frequency	1000 Hz (1ms/tick)
Priority Levels	0 ~ 24 (default 25 levels)
Minimum Stack	768 words (3KB)
Queue/Semaphore/Mutex	All implemented on top of queue
Event Group	24-bit bitmap
Idle Task	One per core (Core 0 IDLE + Core 1 IDLE)
Timer Service Task	Created by kernel automatically, manages all software timers

ADC2 shares resources with Wi-Fi. When Wi-Fi is enabled, ADC2 may be unavailable. Prioritize ADC1 peripherals.

7. Common Concurrency Problems

7.1 Deadlock

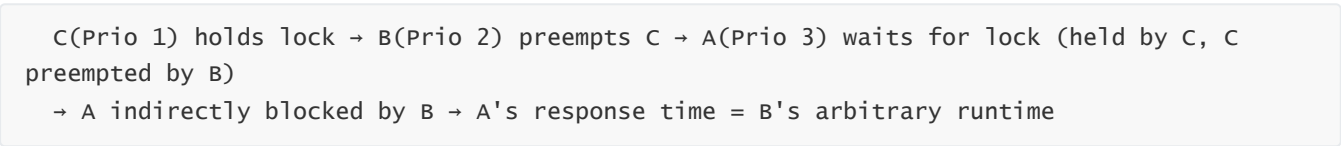
Two tasks waiting for locks held by each other, forming **circular wait** — all related tasks permanently stuck. The following diagram shows a typical deadlock scenario:



Prevention: Unified lock ordering, use timeout, do not call `vTaskDelay` while holding lock.

7.2 Priority Inversion

High priority task is indirectly blocked by waiting for a lock held by a low priority task, while the low priority task is preempted by a medium priority task, causing unpredictable response time for the high priority task.



FreeRTOS mutex has built-in **priority inheritance**, which can shorten inversion time but cannot fundamentally eliminate it. Root cause solution: **reduce lock hold time**.

7.3 Data Race

Multiple tasks simultaneously read/write shared variables without mutex protection, causing data corruption. The following code compares unsafe and mutex-protected approaches:

```
// Dangerous – two tasks modifying g_count simultaneously (non-atomic operation)
// Task B: g_count++; → load → add → store (may overwrite A's result)

// Safe – protected by mutex
xSemaphoreTake(xMutex, portMAX_DELAY); // Take lock
g_count++;                               // critical section
xSemaphoreGive(xMutex);                 // Give lock
```

8. FreeRTOS Configuration in ESP-IDF

FreeRTOS configuration is done via `sdkconfig` or `menuconfig`. Below is the navigation path for key configuration items in menuconfig:

```
idf.py menuconfig
→ Component config
→ FreeRTOS
→ Kernel
→ configMAX_PRIORITIES      (default: 25)
→ configTICK_RATE_HZ        (default: 1000 Hz)
→ configMINIMAL_STACK_SIZE  (default: 768 words)
```

Some configurations can also be overridden via `#define` in code (automatically applied after `sdkconfig.h` is generated).

9. Learning Path Recommendations

The following path is arranged by dependency, earlier experiments are the foundation for later concepts:

- | | |
|--|--|
| 1. RTOS-Task | ← understand what tasks are / create / suspend resume / delete |
| 2. RTOS-QUE | ← understand how tasks pass data (producer-consumer) |
| 3. RTOS-SEM | ← understand how ISR notifies tasks (ISR → task sync) |
| 4. RTOS-MUTEX | ← understand how to protect shared resources (mutual exclusion) |
| 5. RTOS_SCHEDULE
(scheduling rules) | ← understand priority preemption + time slice round-robin |
| 6. RTOS_EVENT_GROUP | ← understand multi-event synchronization (AND/OR wait) |
| 7. RTOS_Timer | ← understand software timer (periodic non-ISR callback) |
| 8. LOW_Power | ← understand deep sleep (lowest power consumption + GPIO wakeup) |
| 9. RTOS_Deadlock | ← understand how deadlock occurs + how to prevent ★ |
| 10. RTOS_Priority_inversion | ← understand how priority inversion occurs + how to prevent ★ |

Not recommended to skip around. Deadlock and priority inversion are placed last for a reason — only after understanding the first 4 communication mechanisms and scheduling rules can you truly understand why these two problems occur and why they are fatal.